

CynLr Problem Statement - Robotics Simulation Design Problem

Advaith Sriram

Abstract—This report details the design and implementation of a decentralized, visual-guided dual-arm synchronization system simulated in PyBullet. The objective is to establish synchronized motion between two robots (UR5 and Franka Emika Panda) using visual feedback as the exclusive communication channel. Robot A picks up a target object and executes a complex, pre-computed 3D trajectory. Robot B tracks the target object using a camera and an axis-wise independent Proportional-Derivative (PD) controller. The system achieved stable, dynamic synchronization, maintaining the Mean Absolute Error (MAE) across all axes below 1.9 cm, despite the high-curvature motion and continuous tracking challenges. This report shows the efficacy of a decentralized, vision-based PD control strategy for complex, dynamic robotic synchronization tasks.

Index Terms—visual tracking, motion generation, synchronization, decentralized control.

I. INTRODUCTION

Modern collaborative robot systems increasingly rely on multi-arm coordination to perform tasks involving manipulation, perception, and interaction with the environment. While traditional approaches frequently employ centralized communication to synchronize multiple manipulators, there is a growing interest in decentralized control systems that rely only on local sensing, such as vision-based feedback. Such systems offer advantages in modularity and scalability, but introduce challenges in achieving real-time coordination without shared information.

This work investigates the problem of visual-guided dual-arm synchronization in a fully simulated environment. The objective is to design and implement a system in which two robotic manipulators operate independently while achieving coordinated behavior through visual feedback alone. The first robot (Robot A) is responsible for object pickup and the execution of a pre-defined motion trajectory, while the second (Robot B) uses an end-effector camera to track the moving object and regulate its pose relative to it. The interaction between the robots is mediated entirely through perception. This report details the implementation of the dual-robot system, focusing on the design and tuning to achieve real-time tracking performance.

II. SYSTEM OVERVIEW

The dual-arm synchronization task was implemented in the PyBullet (Version 3.2.5) simulation environment running on a macOS platform. PyBullet was chosen over alternatives (Gazebo or CoppeliaSim) due to its efficient performance and capability to handle the required physics steps and frequent

kinematic calculations within system constraints. The URDF and mesh files for both robotic manipulators were sourced from a public GitHub repository [1].

The system utilizes two industrial robotic arms:

- 1) Robot A: A UR5 [2] (6-DOF) manipulator responsible for object pickup and complex trajectory execution. After the object pickup is complete, its 3D trajectory is precomputed using Inverse Kinematics (IK) to generate a series of joint angles for smooth execution.
- 2) Robot B: A Franka Emika Panda [3] (7-DOF) manipulator responsible for real-time visual tracking. It has a camera on its end-effector from which RGB-D information is captured. It then uses this information and a PD-based control to follow the UR5. It computes its required joint angles in real-time using IK based on the visual error feedback.

The PyBullet physics engine runs at a rate of 120 Hz. The control and vision loop for Robot B operate at a synchronized frequency of 30 Hz. The control logic is executed immediately following each vision update, ensuring the control response is coupled to the maximum achievable sensor update rate.

Robot B is equipped with a simulated RGB-D camera mounted directly on its end-effector. The camera is offset by 15 cm and pointed outwards to ensure that the robot's own structure is not visible in the image frame. This allows for a clear view of the target object. The camera was configured with a resolution of 320×240 pixels, a field of view (FOV) of 60° , and clipping planes of 0.01m and 5.0m for near and far respectively. The resolution in particular was chosen as a trade-off to reduce the computational load associated with image processing and control updates.

III. METHOD

A. Simulation Environment Setup

The simulation environment was constructed in PyBullet to provide a defined workspace for the dual-arm task. Both Robot A (path executor) and Robot B (follower) were set up on a table. A cube was placed on the table of Robot A for the object pickup sequence. The component placements are described below in Table I and is shown in Figure 1.

B. Object Pickup Sequence

Robot A initiates the process by retrieving the target object (cube) from its starting position. Instead of using

TABLE I: Simulation Environment Component Placement

Component	Position [X, Y, Z] [m]	Size [m]
Table A (Center)	[0.50, 0.00, 0.25]	$1.00 \times 1.00 \times 0.50$
Robot A (UR5) Base	[0.50, 0.00, 0.52]	-
Table B (Center)	[2.00, 0.00, 0.25]	$1.00 \times 1.00 \times 0.50$
Robot B (Franka) Base	[2.00, 0.20, 0.52]	-
Pickup Object (Cube)	[0.80, -0.30, 0.52]	$0.04 \times 0.04 \times 0.04$

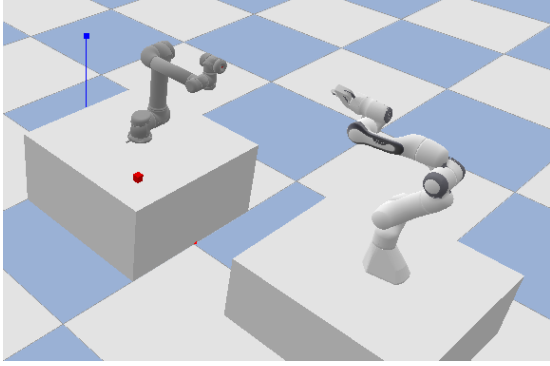


Fig. 1: PyBullet Environment Setup illustrating the configuration of Robot A (UR5), Robot B (Franka Emika Panda), the target object (red cube), and the tables (white).

a computationally expensive Cartesian path planning, the system uses joint-space interpolation, which generates joint angle waypoints between the current and goal joint angle configuration. It is computationally efficient as the IK is only called twice (start and end). The end-effector path however, is non-linear.

The robot moves to a hover position of 5 cm above the object (with the end-effector pointed downwards), and then moves down vertically to the object. The object is attached to the end-effector, after which it moves vertically upward back to the hover position.. The arm then moves back to its initial position, ready to begin the 3D trajectory.

C. Simplified Grasping and Justification

In the current version of the project, once the end-effector moves down vertically to the object (from the hover position), a simplified grasping operation was performed. The PyBullet function `createConstraint` was used to establish a fixed constraint between the end effector link of Robot A and the object's base link. This simulates a perfect, rigid grasp, an example of which is shown in Figure 2.

This fixed constraint serves as an abstraction for simple, non-actuated grasping scenarios where a *physical gripper is not required*. This setup models grippers that achieve an instantaneous, rigid connection, such as suction cups, magnetic end-effectors, or hook-and-loop fasteners (Velcro-based attachments). Using this abstraction allows us to bypass complex force control and contact physics. Using a fixed constraint also eliminates potential sources of simulation instability (e.g.,

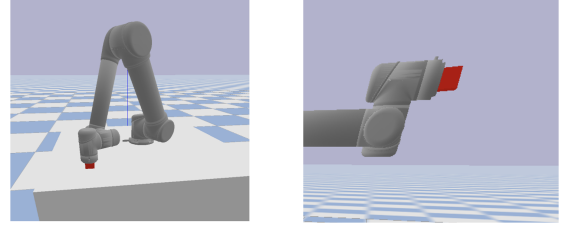


Fig. 2: Simplified grasping operation simulating a perfect, rigid grasp

object slippage, failed grasp) that would affect the reliability of the main trajectory tracking.

D. Motion Generation and Trajectory Execution

Following the object pickup, Robot A (UR5) executes a pre-defined, continuous 3D motion pattern. The overall motion is defined by two alternating, planar patterns in the YZ -plane (Circular and Lissajous), overlaid with a sinusoidal motion along the perpendicular X -axis.

The constant parameters defining the size and speed of the trajectory are summarized in Table II.

TABLE II: Pre-Defined Trajectory Parameters

Parameter	Symbol	Value
YZ -Plane Radius/Amplitude	R, A_y, A_z	0.125 m
X -Axis Sinusoidal Amplitude	A_{sin}	0.08 m
Frequency	f_{sin}, f	1.0 cycle per trajectory

The target position $\mathbf{P}(t) = [X(t), Y(t), Z(t)]^T$ is calculated based on the current time t . $\mathbf{P}_c = [X_c, Y_c, Z_c]^T$ denotes the center position of the trajectory. $\omega = 2\pi f$ is the angular frequency.

1) *Circular Trajectory*: The circular phase performs a counter-clockwise motion in the YZ -plane.

$$X(t) = X_c + A_{sin} \sin(\omega_{sin} t) \quad (1)$$

$$Y(t) = Y_c + R \cos(\omega t) \quad (2)$$

$$Z(t) = Z_c + R \sin(\omega t) \quad (3)$$

2) *Lissajous Trajectory (Infinity)*: The Lissajous phase traces a horizontal figure-eight (or infinity symbol) in the YZ -plane, achieved through a 1 : 2 frequency ratio between the Y and Z components.

$$X(t) = X_c + A_{sin} \sin(\omega_{sin} t) \quad (4)$$

$$Y(t) = Y_c + A_y \sin(\omega t + \frac{\pi}{2}) \quad (5)$$

$$Z(t) = Z_c - A_z \sin(2\omega t + \pi) \quad (6)$$

The trajectory for Robot A is pre-computed to minimize run-time latency and ensure smooth execution. The entire trajectory is discretized into a sequence of $N = 200$ Cartesian waypoints. For each waypoint $\mathbf{P}(t_i)$, the required joint configuration θ_i is calculated using the PyBullet IK solver (`calculateInverseKinematics`) before the trajectory

execution starts, to ensure that the kinematics is solved outside of the simulation loop. The motion between adjacent waypoints is governed by the joint controller's position gain (of 0.3). This results in a smooth motion profile that is repeatable, resulting in a constant input for Robot B. The trajectory motion generated by Robot A along the YZ-plane, X-axis and in 3D is shown in Figure 3.

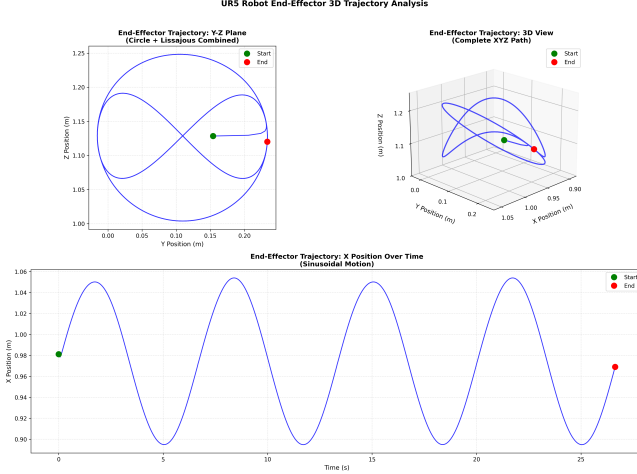


Fig. 3: YZ-plane (Circle and Lissajous curve) (top-left), 3D Trajectory (top-right), and X-axis (Sinusoidal) over time (bottom) of Robot A

E. Perception and Control

Once Robot A begins performing the pre-defined trajectory, Robot B begins its visual tracking of the object (cube). A visualization of the camera footage is shown in Figure 4. Using the RGB-D camera, it calculates the 3D error vector $e(t)$ of the target object relative to the camera center. The RGB image is processed using color thresholding in the HSV space to segment out the red object from the background. The centroid of the largest segment provides the 2D pixel coordinates in u, v . Using the depth map, the depth d is extracted at the 2D pixel coordinates. The error $e = [e_x, e_y, e_z]^T$ (with x, y, z representing u, v, d respectively) is calculated, and a Low-Pass Filter (LPF) with a smoothing factor (α) set to 0.7 is applied to reduce noise before being passed to the controller. A dead-band is applied to the errors to prevent minute corrections when the error is negligible.



Fig. 4: Simulated Camera Footage from Robot B's end effector shown in PyBullet

Robot B uses a Proportional-Derivative (PD) control law to generate the required corrective displacement in the camera's Cartesian space, with the objective to drive the 3D error vector e to 0. The discretized control command can be given by the following equation, where $i \in \{x, y, z\}$:

$$\Delta_{i,cam} = K_{P,i}e_i + K_{D,i}(e_i - e_{prev,i}) \quad (7)$$

These control gains were tuned and determined empirically to achieve stability and minimize tracking lag. The final tuned gains used are detailed in Table III

TABLE III: Robot B PD Controller Gains

Axis	P-Gain (K_P)	D-Gain (K_D)
X (Camera Right/Left)	0.001	0.0005
Y (Camera Up/Down)	0.001	0.0005
Z (Camera Depth)	0.25	0.04

The resulting displacement vector Δ_{cam} is in the camera's local coordinate frame. This was transformed into the PyBullet world frame using the following equation, where $\mathbf{P}_{target}$ represents the end-effector target position, $\mathbf{P}_{current}$ represents the current end-effector position, and $\mathbf{R}_{W \leftarrow C}$ represents the rotation matrix from the camera frame to the world frame:

$$\mathbf{P}_{target} = \mathbf{P}_{current} + \mathbf{R}_{W \leftarrow C} \Delta_{cam}$$

$\mathbf{P}_{target}$ is then passed to the real-time IK solver to determine the joint position commands θ_{target} .

IV. RESULTS

The visual tracking performance achieved by Robot B under the decentralized PD control strategy is analyzed. The overall task execution, where Robot B tracks the complex 3D motion of Robot A's end-effector, is illustrated by the trajectory overlay plots in Figure 5. This figure displays the tracking results for the first *two full cycles* of the combined trajectory (Circle $\rightarrow$ Lissajous $\rightarrow$ Circle $\rightarrow$ Lissajous) after the PD gains have been tuned. This, along with the video demonstration, shows a qualitative assessment of the tracking.

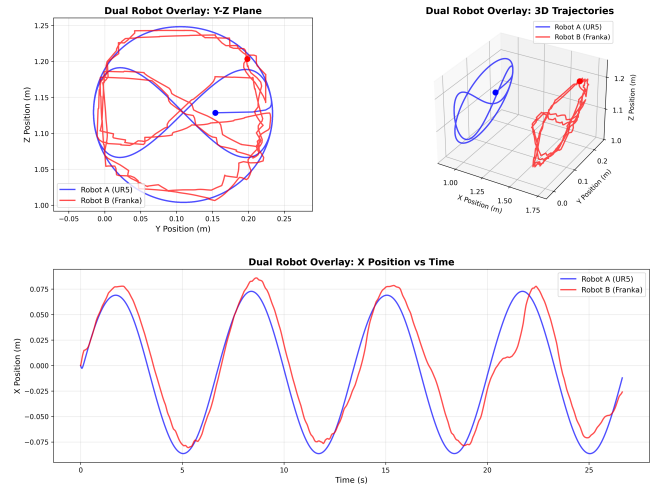


Fig. 5: Overlay of Robot B tracking (red) on Robot A's pre-defined trajectory (blue)

As shown in the **Y-Z Plane** plot, Robot B, shown in **red**, successfully follows the alternating circular and Lissajous patterns defined by Robot A, shown in **blue**. The tracking motion is generally smooth, but exhibits oscillatory deviations, especially along the tightly curved sections of the Lissajous pattern. The **X Position vs Time** plot, representing the depth control, confirms that Robot B is able to track the sinusoidal motion of Robot A. Through this, the relative distance between the two end-effectors is maintained.

The quantitative performance is evaluated using the instantaneous absolute error between the end-effector positions of Robot A and Robot B, $E_{abs}(t) = |\mathbf{P}_B(t) - \mathbf{P}_A(t)|$. Figure 6 shows the instantaneous absolute tracking error (in meters) in the global X , Y , and Z axes throughout the simulation.

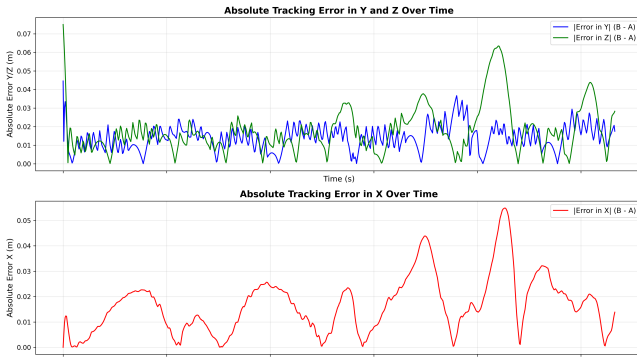


Fig. 6: Instantaneous tracking error (absolute) of Robot B

The errors in the Y and Z axes (top) are generally contained below 0.03 m (3 cm), with the highest peaks occurring when Robot A transitions between the different trajectory segments (circle to Lissajous). The **X-axis (Depth) error** (bottom) shows a larger magnitude, peaking near 0.05 m, particularly around the 18 and 22 second marks. This higher error in depth is consistent with the greater difficulty in accurately controlling motion along the line of sight using the depth camera.

TABLE IV: Robot B Tracking Performance Metrics

Control Axis (Global)	Mean Absolute Error (MAE) [m]	Root Mean Squared Error (RMSE) [m]
X (Depth)	0.01689	0.02037
Y (Horizontal)	0.01406	0.01542
Z (Vertical)	0.01807	0.02196

The overall tracking performance, summarized by the MAE and RMSE in Table IV, confirmed effective synchronization with all axis errors maintained below **2.2 cm**.

V. DISCUSSION AND CONCLUSION

The results validate the efficacy of the decentralized, vision-based PD control system for the dual-arm synchronization. Robot B successfully tracked the complex 3D trajectory of

Robot A. The performance varied across the controlled axes, leading to key observations.

- **Planar vs. Depth Control:** The **Y-axis (Horizontal)** achieved the lowest error (MAE = 0.01406 m). In contrast, the **Z-axis (Vertical)** exhibited the highest RMSE (0.02196 m), suggesting that there were more fluctuations in the vertical component, potentially due to the influence of gravity, or compounded errors. Additionally, the high error observed in the **X-axis (Depth)** is consistent with the difficulty in accurately controlling motion along the camera's line of sight using depth sensing.
- **Response to High Curvature:** Qualitative assessment of the overlay showed that the most pronounced oscillatory deviations occurred following the transition from the circular to the Lissajous trajectories. This instability is primarily due to the controller's lag and the high curvature changes required that the PD controller could not instantaneously manage without overshoot.
- **Long-Term Stability:** An observation regarding the robustness was the gradual increase in oscillatory deviations with subsequent cycles of the combined trajectory. This suggests a long-term instability or accumulation of small, uncorrected tracking errors. This accumulating error is a limitation of the PD control approach and future work should prioritize the implementation of a full Proportional-Integral-Derivative (PID) controller.
- **Impact of 6-DOF Pose Constraint:** All 6 axes of Robot B's end effector had to be controlled to track the target object while maintaining a fixed camera pose relative to it. The Franka Emika Panda, with its 7th DOF, provided necessary kinematic redundancy to avoid singularities [4] and optimize joint limits. However, this full 6-DOF constraint led to significant dynamic coupling, which contributed to the overall system shakiness. This is supported by comparative trials, where relaxing the global X -axis (depth) control resulted in noticeably smoother tracking along the global YZ -plane. This is shown in Appendix: Figure 8.

In conclusion, this work successfully implemented and validated a decentralized visual-based tracking system for dual-arm synchronization in a simulated environment. By parallelizing the vision processing pipeline and using an axis-wise independent PD control for the camera axes, the system was able to track the 3D target trajectory. The measured MAE across all axes was below 1.9 cm, confirming the effectiveness of the control strategy.

REFERENCES

- [1] ElectronicElephant, "pybullet_ur5_robotiq (robotflow branch)," https://github.com/ElectronicElephant/pybullet_ur5_robotiq/tree/robotflow, 2025, gitHub repository.
- [2] U. Robots, "Ur5 technical specifications," https://www.universal-robots.com/media/50588/ur5_en.pdf, Universal Robots, 2016, datasheet.
- [3] F. Emika, "Panda – robot arm control," https://pkj-robotics.dk/wp-content/uploads/2020/09/Franka-Emika_Brochure_EN_April20_PKJ.pdf, Franka Emika, 2020, datasheet.
- [4] F. Beck, M. N. Vu, C. Hartl-Nesic, and A. Kugi, "Singularity avoidance with application to online trajectory optimization for serial manipulators," *arXiv (Cornell University)*, Nov 2022.

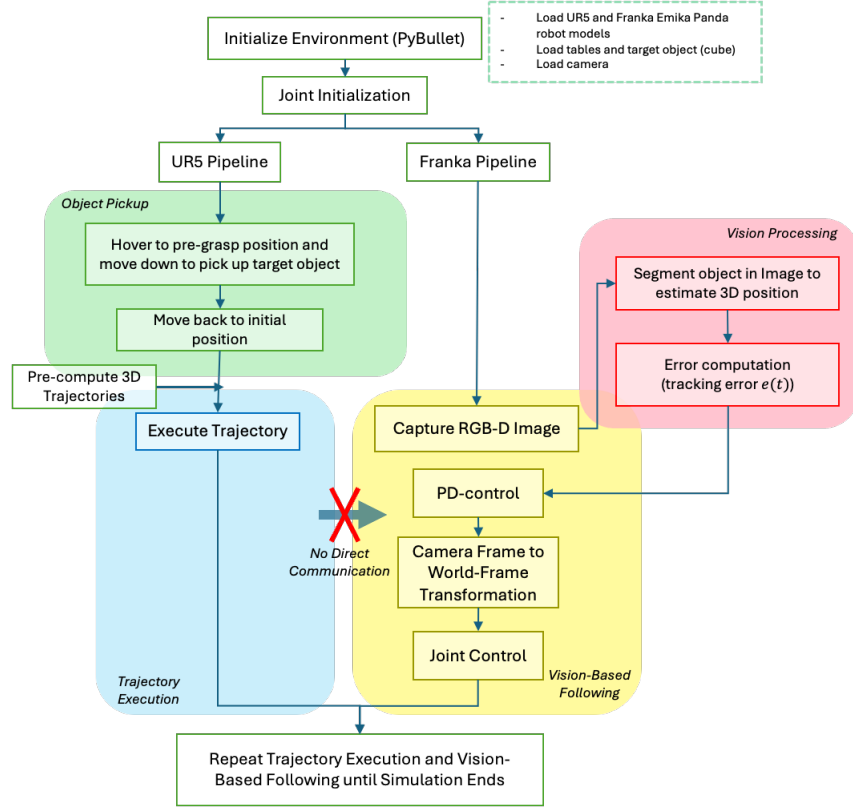


Fig. 7: System-level flowchart of the decentralized dual-arm simulation architecture

TABLE V: Comparative Robot B Tracking Performance (Depth Control Relaxed)

Control Axis	MAE [m]	RMSE [m]
X (Depth, Uncontrolled)	0.05364	0.06199
Y (Horizontal, Controlled)	0.01526	0.01699
Z (Vertical, Controlled)	0.01642	0.01886

APPENDIX

A. Flowchart

Figure 7 shows the system-level flowchart of the decentralized dual-arm simulation architecture. Robot A (UR5) performs object pickup, transfer, and execution of pre-computed 3D trajectories. Robot B (Franka Emika Panda) tracks the moving object using only RGB-D images from an end-effector-mounted camera, performing segmentation, 3D pose estimation, error computation, and PD-based visual tracking. No joint states or control signals are exchanged between robots. All synchronization occurs exclusively through the visual feedback loop.

B. Tracking under Reduced Constraint (Free Depth Axis)

Figure 8 and Table V show the tracking performance when the global X -axis (depth) control was deactivated. The increased smoothness in the YZ planar trajectories (compared to when all axes were fixed) confirms the impact of depth control coupling on the system.

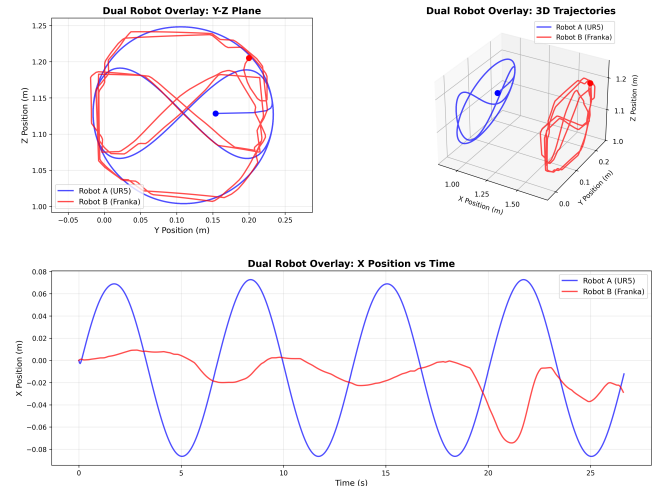


Fig. 8: Depth Control Relaxed - Overlay of Robot B tracking (red) on Robot A's pre-defined trajectory (blue)